The QueryGraph Stack

The Definitive Guide to the Governed Semantic Lakehouse

Stack release 0.4.0-bf45521 built 2026-07-05 — troff edition

Alexy Khrabrov and Slava Tykhonov
querygraph.ai

Executive Summary

The QueryGraph stack is a governed semantic lakehouse for agentic AI: five coordinated open-source components that let AI agents answer questions over enterprise data while *proving* what they did — which semantics they used, which policies allowed it, which sources they touched, and which they were denied. The stack's thesis is that the differentiating infrastructure for enterprise AI is not a bigger model or a longer context window, but a verifiable chain from question to answer: deterministic identity, dual policy gating, signed envelopes, canonical hashing, and lineage anchoring, all projected through open standards.

The five components, each independently useful, released in coordinated, codenamed versions:

- **Grust** — a backend-neutral property-graph API for Rust with a standards-conformant GQL/Cypher read surface and a dozen storage backends. The semantic graph substrate. (Part I, Chapters 1–2.)
- **TypeSec** — agentic AI security in Rust's type system: unforgeable capabilities, TypeDID signed agent envelopes, audit events, and — since Torcello — a security platform other agent stacks plug into. (Part I, Chapters 3–4.)
- **LakeCat** — a Rust-native Apache Iceberg REST catalog that binds catalog state, governed Sail planning, TypeSec receipts, and Grust projection to the same table transitions. The catalog boundary. (Part I, Chapters 5–6.)
- **Sail** (fork) — the Spark-compatible compute engine, extended with a Cypher graph-query surface compiled into its SQL frontend. The lakehouse engine. (Part I, Chapter 7.)
- **QueryGraph** — the semantic layer itself, in Rust (`qg-rust`) and Python (`qg-python`): four semantic projections (Semantic Croissant, CDIF, W3C DID, ODRL) plus OSI business semantics, RBAC+ODRL dual gating, Ed25519-signed TypeDID envelopes verified across languages, OpenLineage audit with signed attestations, an HTTP `/v1` API with envelope authentication, MCP servers in both languages, an A2A agent card, and the governed navigator loop. (Parts II–III.)

Part IV walks the integrations end to end — catalog to governed answer, agent frameworks plugging in, and operating the stack — and the closing chapter lays out future work.

As of this guide, the current releases are **QueryGraph 0.4.0 "Sentinel"** (the governed-answer release, in both languages, following 0.3.0 "Goshawk", the interoperability release), over **Grust 0.12.0 "Lobster"**, **TypeSec 0.12.0 "Torcello"**, and **LakeCat 0.3.0 "Ocelot"** — a coordinated substrate wave in which Grust merged its Full39075 GQL goal, TypeSec became an agent-interoperability security platform, and LakeCat proved stock-client Iceberg REST conformance.

Links

What	Where
Workspace meta-repo (map, review, this guide's home)	https://github.com/querygraph/querygraph
QueryGraph Rust (`qg-rust`)	https://github.com/querygraph/qg-rust
QueryGraph Python (`qg-python`)	https://github.com/querygraph/qg-python
Grust	https://github.com/querygraph/grust
TypeSec	https://github.com/querygraph/typesec
LakeCat	https://github.com/querygraph/lakecat
Sail upstream (forked with the Cypher extension)	https://github.com/lakehq/sail
Sentinel releases	https://github.com/querygraph/qg-rust/releases/tag/v0.4.0 · https://github.com/querygraph/qg-python/releases/tag/v0.4.0
The dedicated QueryGraph book	`qg-rust/docs/book`
Semantic Croissant (MLCommons Croissant)	https://mlcommons.org/croissant/
CDIF (CODATA Cross-Domain Interoperability Framework)	https://cdif.codata.org/

W3C DID / ODRL	https://www.w3.org/TR/did-core/ · https://www.w3.org/TR/odrl-model/
OSI (Open Semantic Interchange)	https://github.com/open-semantic-interchange
OpenLineage	https://openlineage.io/
Model Context Protocol	https://modelcontextprotocol.io/
Agent2Agent (A2A)	https://github.com/a2aproject/A2A

This guide is the stack-wide companion to the dedicated QueryGraph book (which walks the semantic layer itself as a textbook). It is written to be self-contained: each component gets its own part and chapters, each standard is explained before it is used, every chapter closes with worked examples in Rust and Python as applicable, and the outputs shown are captured from real runs against the released code.

The Stack: An Overview

The journey of a question

A question enters the stack at the top and evidence comes out at the bottom:

1. An agent — or a person, or another agent framework speaking MCP or HTTP — asks a question.
2. QueryGraph searches the **semantic model** — OSI business terms, Croissant field semantics, ontology terms — for what the question means *here*, in this organization's vocabulary.
3. The **rights layer** decides, per source, whether this principal may read it: RBAC *and* ODRL must both allow. Every decision, allow or deny, becomes a receipt.
4. SQL is planned only over allowed **Sail** sources; denied sources are named as off-limits, and never touched.
5. Synthesis happens — deterministically, or via any LLM — and the answer travels in a **TypeDID envelope** signed with Ed25519 under a `did:key` verification method.
6. An **OpenLineage** event (spec-conformant, schema-validated) records the run, and an Ed25519 **attestation** anchors the event hash.

Each step is somebody's component. The semantic graph lives in Grust (and is queryable through Sail's Cypher extension). The envelope machinery and the capability model are TypeSec. The tables come from a lakehouse whose catalog transitions LakeCat governs. And QueryGraph is the layer that makes them one system with one evidence chain.

A useful way to read that chain — the sentence the whole stack exists to make true: *the answer used OSI metric X, resolved to Sail table Y, under capability C and `odrl:read`, emitting OpenLineage run R anchored by attestation A — and source Z was denied, with a receipt.* No mainstream agent framework offers that sentence.

Versions and codenames

Stack releases are coordinated by codename, each repo keeping SemVer independently:

Component	Version	Codename	Pool
QueryGraph (both languages)	0.4.0	Sentinel	birds of prey
Grust	0.12.0	Lobster	—
TypeSec	0.12.0	Torcello	Venetian landmarks
LakeCat	0.3.0	Ocelot	wild cats

The 0.12 substrate wave landed together: Grust merged the Full39075 GQL profile, TypeSec shipped its agent-interoperability platform, and LakeCat moved to both while proving stock-client Iceberg REST conformance — with QueryGraph verified green against all three. QueryGraph 0.4.0 "Sentinel" ships on top of that wave: where Goshawk opened the doors (MCP, A2A, `/v1`, cross-language crypto), Sentinel stands guard over what comes through them.

The workspace

The repositories expect sibling checkouts:

```
~/src/
 querygraph/           # meta-repo: README, FABLE-REVIEW-1, workspace map
|   qg-rust/          # reference implementation (this guide lives here)
|   qg-python/        # Pythonic mirror
|   sail/             # fork, branch `grust`, Cypher extension
|   semantic/         # research: Polaris SemanticModel, OSI round-trips
 grust/                # path dependency of qg-rust
 lakecat/              # path dependency of qg-rust
 typesec/              # consumed from crates.io
```

Part I: The Substrate

The four components underneath the semantic layer: the graph and its query language, the security fabric and its identity machinery, the catalog and its handoff, and the compute engine. Each chapter explains its component from scratch and ends with worked examples.

Chapter 1. Grust: The Property Graph

Grust is a modern property-graph API for Rust with a deliberately plain core model:

```
Graph = nodes + edges
Node  = id + label + properties
Edge  = optional id + from + to + label + properties
```

That shape is expressive enough for persistent graph databases but small enough for tests, import/export tools, scrapers, and knowledge-graph pipelines. The design principle is strict separation of concerns: application code builds a `grust::Graph`; backend crates decide how to persist or query it. Grust is not competing with `petgraph` for in-memory graph algorithms — it is the *persistent property-graph layer*: stable application IDs, node and edge labels, typed properties, optional schema metadata, traversal expressed as an IR rather than a query string, and an async `GraphStore` trait for persistence backends.

The workspace ships a facade crate (`grust-graph`, imported as `grust`) over a core (model, builder, schema validation, traversal IR, `GraphStore`) and a family of backends:

Backend crate	Target
`grust-memory`	deterministic in-memory store for tests and local use
`grust-sail`	**Sail SparkConnect** — graphs as Spark DataFrames (QueryGraph's choice)
`grust-turso`	embedded Turso/SQLite storage (LakeCat's choice)
`grust-postgres`, `-postgres-pgq`, `-pggraph`	PostgreSQL: universal tables, SQL/PGQ, pgGraph
`grust-surreal`, `grust-helix`, `grust-falkor`, `grust-ladybug`, `grust-lancedb`	SurrealDB, HelixDB, FalkorDB, LadybugDB, LanceDB
`grust-cocoindex`	CocoIndex-style target-state export

The builder deduplicates nodes by id and, by default, edges by `(from, label, to)`; domains that need multi-edges opt into `EdgePolicy::AllowDuplicates`. Typed property schemas validate graphs before they reach a store.

QueryGraph uses Grust twice. First, the semantic graph — datasets, fields, ontology terms, agents, policies, as nodes and edges — loads into a Grust store. Second, the Sail fork compiles a Cypher extension *into* the engine (Chapter 7), reusing Grust's model, so the same graph is queryable from Spark sessions.

Worked example: build a graph, store it, traverse it

```
use grust::prelude::*;

let mut builder = GraphBuilder::new();
let dataset = builder
    .node("Dataset", "dataset:county-finance")
    .prop("source", "sail.qg_lakehouse.government_finance__countydata")
    .finish();
let metric = builder
    .node("Metric", "metric:fiscal-capacity")
    .prop("expression", "SUM(total_revenue - mandated_spend)")
    .finish();
builder.edge("MEASURED_OVER", &metric, &dataset).finish();
let graph = builder.build();

// Persist and traverse (memory store; SailGraphStore is the same trait).
let store = MemoryGraphStore::new();
store.put_graph(&graph).await?;
let datasets = store
    .traverse(
        Traversal::from_node("metric:fiscal-capacity")
            .out("MEASURED_OVER")
            .to("Dataset"),
    )
    .await?;
assert_eq!(datasets.len(), 1);
```

The traversal is an IR, not a string: the same `Traversal` runs against memory, PostgreSQL, or Sail, each backend lowering it natively.

API reference

Surface	Essentials
`GraphBuilder`	`new()`, `.node(label, id).prop(k, v).finish()`, `.edge(label, &from, &to).prop(k, v).finish()`, `.edge_policy(EdgePolicy::AllowDuplicates)`, `.build() -> Graph`
`Graph` / `Node` / `Edge`	plain data: ids, labels, typed property maps; serde-serializable
`Traversal`	`from_node(id).out(label).to(label)` — an IR, lowered natively per backend
`GraphStore` (async trait)	`put_node`, `put_edge`, `put_graph`, `get_node`, `traverse`
Stores	`MemoryGraphStore::new()`; `SailGraphStore` (Spark Connect); `grust-turso`, `grust-postgres{,-pgq}`, `grust-surreal`, `grust-helix`, `grust-falkor`, `grust-ladybug`, `grust-lancedb`

Schema	optional typed property schemas; graphs validate before reaching a store

Chapter 2. The Query Language: GQL/Cypher

Crab (0.11) gave the graph a language: a standards-conformant GQL/Cypher layer — lexer, parser, AST, semantic analysis — over the property graph, with read pushdown into Sail and SQLite, first-class Decimal/Duration/temporal values, and catalog procedures such as `CALL db.labels()`.

Lobster (0.12) completes it. The merged **Full39075 profile** brings:

- `CALL { ... }` subqueries;
- table-valued functions (`CALL` with correlated arguments);
- `shortestPath()` / `allShortestPaths()` on the read reference;
- backend-native query passthrough escape hatches;
- an executable portable read corpus (the conformance tests run as code);
- **atomic Cypher transaction batches**: `CypherTransaction` accumulates eagerly-planned write statements between `START TRANSACTION`/`BEGIN` and commit, executing them atomically behind the transaction surface.

Reads are portable: `grust-cypher::read::run_read_query` executes Cypher against any `grust::Graph` with a reference executor, while stores like `SailGraphStore` push the identical query down to the backend. Writable Cypher is a strict, backend-neutral mutation plan (`cypher_mutation_plan`) that stores execute natively.

Worked example: Cypher over the semantic graph

Exactly how qg-rust queries its semantic graph (`src/cypher.rs`):

```
use grust_cypher::read::run_read_query;
use grust_cypher::CypherParameters;

let table = run_read_query(
    &graph,
    "MATCH (m:Metric)-[:MEASURED_OVER]->(d:Dataset) \
     RETURN m.expression, d.source",
    &CypherParameters::new(),
)?;
```

Through the Sail fork's extension (Chapter 7), the same `MATCH` also runs from any Spark Connect session — PySpark included — against the graph the lakehouse projects.

API reference

Surface	Essentials
`grust_cypher::read::run_read_query(&graph, cypher, &params)`	portable reference executor over any `grust::Graph`; returns `CypherResultTable`
`CypherParameters`	`new()`, typed parameter binding
`cypher_mutation_plan(cypher)` / `sail_cypher_mutation_plan`	strict backend-neutral write plans
`SailGraphStore::run_read_query` / `execute_cypher_mutation*`	backend pushdown; `_returning_with_options` returns generated ids
`CypherTransaction`	accumulates eagerly-planned writes between `BEGIN`/`START TRANSACTION` and commit; atomic execution

`CypherSession` / `SessionCommand`	standalone `USE` and session commands
Catalog procedures	`CALL db.labels()` and friends, over the projected graph

Chapter 3. TypeSec: Capabilities as Types

TypeSec encodes permissions as types. Most security systems check permissions at runtime — and a guard-based check can be forgotten, skipped, or bypassed:

```
//  Guard-based — one missed call and the policy is fiction.
if acl.check(user, "write", resource) {
    resource.write(data);
}
```

TypeSec inverts this. If your code does not hold a `Capability<CanWrite, Report>`, the write method *does not exist* for you:

```
//  Type-level — the capability IS the proof.
fn write(cap: Capability<CanWrite, Report>, report: &Report) {
    // `cap` existing in scope means the policy engine approved this.
}
```

The `Capability<P, R>` struct is unforgeable by construction:

- its constructor is `pub(crate)` — only the policy engine creates one;
- `P` and `R` are phantom types — `Capability<CanRead, Report>` and `Capability<CanWrite, Report>` are *different types*;
- the `Permission` trait is sealed — no new permissions outside `typesec-core`.

The only production path to a capability runs a policy check and emits an audit event; a denial is a typed error carrying a receipt, never a capability. Policy violations are compile errors, not incident reports.

QueryGraph's supervised agent story (Part II, Chapter 16) mints capabilities for exactly the actions its topology needs: `AiCanInfer`, `CanReadSensitive`, `CanDelegate`, `CanAggregateSummaries`, `CanReadDatasetCompartment`, and `CanDeriveRedactedSummary`.

Worked example: a capability is the proof

```
use typesec::prelude::*;

// Without a Capability<CanRead, Report> in hand, this function
// cannot be called — the check is the type system's, not yours.
fn read_report(cap: Capability<CanRead, Report>, report: &Report) -> Summary {
    summarize(report) // `cap` in scope == the policy engine said yes
}

let decision = engine.check(&subject, Action::Read, &report);
match decision {
    Ok(cap) => read_report(cap, &report),      // audit event already emitted
    Err(denied) => return Err(denied.receipt()) // a denial is data, not panic
};
```

API reference

Surface	Essentials
`Capability<P, R>`	unforgeable proof; `pub(crate)` constructor; phantom permission/resource types

`Permission` (sealed)	e.g. `CanRead`, `CanWrite`; QueryGraph adds domain permissions (`AiCanInfer`, `CanReadSensitive`, `CanDelegate`, `CanAggregateSummaries`, `CanReadDatasetCompartment`, `CanDeriveRedactedSummary`)
`PolicyEngine::check(subject, action, resource)`	`Ok(Capability)` — audit event emitted — or `Err(Denied)` with a receipt
`Capability::<P, R>::permission_name()`	stable permission strings for evidence reports
Policy engines	RBAC, ODRL, and graph policy backends behind one `check`

Chapter 4. TypeDID: Identity, Envelopes, and the Torcello Platform

On top of the capability core, TypeSec provides the machinery QueryGraph uses for agent identity and messaging:

- **Deterministic Ed25519 keys**: `Ed25519DidKey::from_seed` derives a keypair from a seed (SHA-256 of the seed as the private key), so an agent recreated from the same seed signs identically across processes — and, as QueryGraph proves, across languages.
- **`did:key` documents**: public keys published as W3C DID identifiers anyone can resolve without a registry.
- **Signed, encrypted envelopes** under the `ed25519-x25519-chacha20` profile, with a request/reply conversation model (the `typedid/a2a` protocol label — made literal by QueryGraph's A2A card, Part III).
- **Audit-safe attestations**: each verified envelope exposes who did what to which resource at which privacy level — without revealing the payload or the signing material. The envelope digest binds the attestation to the exact message.

Torcello (0.12) grows this fabric into a platform other agent stacks plug into: an agent-framework **interop plane** guarding OpenAI, Anthropic, LangChain, and Pydantic-AI tool calls; an MCP dialect and **`mcp-gate`**, a deny-by-default MCP stdio proxy; **signed decision receipts** with decision logging and replay; JSON-Schema-validated tool bindings and a `#[typesec_tool]` macro; an OpenTelemetry audit sink; a WASM decision core for JS/TS agents; a PyPI-ready Python package; and an OpenAI/Anthropic-compatible **enforcement proxy** — the "governed inference proxy" pattern, built at the security layer. QueryGraph already builds against Torcello; adopting these surfaces rather than duplicating them is the next integration step (see Future Work).

Worked example: one seed, one identity, two languages

Python signs:

```
from querygraph.typedid import TypeDidAgent

supervisor = TypeDidAgent.new("SupervisorAgent")
envelope = supervisor.request(
    TypeDidAgent.new("FinanceAgent"),
    action="summarize", resource="compartment:finance",
    payload={"question": "Where is fiscal stress highest?"},
)
print(envelope.signature[:24])            # ed25519:8a7a231b6f67f4...
print(envelope.verification_method[:32])  # did:key:z6Mkrdhpo...
```

Rust mints the identical identity from the identical seed:

```
use querygraph::agent::PyTypeDidEnvelope;

let envelope = PyTypeDidEnvelope::signed(
    "querygraph-agent:SupervisorAgent",   // same seed ⇒ same did:key
    "did:example:recipient",
    "summarize", "compartment:finance",
    serde_json::json!({"question": "Where is fiscal stress highest?"}),
```

```
);
assert!(envelope.verify().signature_valid);
```

A fixture test pins the shared `did:key`, so the derivations can never drift.

API reference

Surface	Essentials
`Ed25519DidKey::from_seed(bytes)`	deterministic keypair — sha256(seed) as private key
`Did::key(signing_public)`	`did:key:z6Mk...` identifier
`TypeDidProfile::ed25519_x25519_chacha20()`	the negotiated envelope profile
`A2aTypeDidAdapter::wrap(TypeDidWrapRequest, resolver, key_store)`	sign + encrypt a payload into an envelope
`TypeDidGateway::open_message(&envelope)`	verify + decrypt; yields `VerifiedTypeDidMessage`
`VerifiedTypeDidMessage::attestation()`	audit-safe: action, resource, privacy, profile, envelope digest
Torcello surfaces	interop plane (OpenAI/Anthropic/LangChain/Pydantic-AI guards), `mcp-gate`, signed decision receipts + replay, `#[typesec_tool]`, OTel sink, WASM core, enforcement proxy, PyPI `typesec`

Chapter 5. LakeCat: The Iceberg REST Catalog

LakeCat is a Rust-native Apache Iceberg REST catalog and QueryGraph foundation. Standard Iceberg clients speak to it on ordinary REST catalog paths (`/catalog/v1`); underneath, it binds catalog state, governed Sail planning, TypeSec receipts, and Grust projection to the *same accepted table transition*, so the catalog's view of a table and the governance evidence about it can never drift apart.

The catalog spine runs on Turso MVCC: commits to different tables run truly concurrently, and a same-table race converges to exactly one winner through a pointer compare-and-swap — no global write lock. The audit event, the lineage outbox row, and the idempotency record are written in the *same transaction* as the table change, which is what lets a downstream consumer accept catalog state as proof rather than as a best-effort side effect.

Scan planning routes through the Sail-facing engine: point-in-time scans produce opaque Iceberg REST plan-task tokens from stable Sail metadata; append-only incremental scans over a snapshot chain plan only the manifests added in the requested range, expanding delete manifests through Sail's delete-file index. REST scan filters are validated against Sail's generated Iceberg expression models before planning.

Ocelot (0.3.0) proves the boundary from the stock client's side: Iceberg REST conformance demonstrated by a **PyIceberg round-trip** — spec-correct error types (403 on authorization denial, 409 on a duplicate namespace, 404 on a missing one), `listTables`, honest update rejection on the default build, and fail-closed commit-requirement validation — over dependencies moved to Grust "Lobster" and TypeSec "Torcello". The release rests on a recorded release-candidate proof harness (Chapter 6's handoff, verified end to end, 40/40 QueryGraph tests green).

API reference

Surface	Essentials
`GET /catalog/v1/config`	standard Iceberg REST config — stock clients start here
`/catalog/v1/namespaces...`	namespace CRUD, `listTables`, table load/commit — Iceberg-spec error model (403/404/409 with spec `type` strings)

Scan planning	point-in-time and append-only incremental plans as opaque plan-task tokens from stable Sail metadata; filters validated against generated Iceberg expression models
`GET /querygraph/v1/bootstrap`	the bootstrap bundle: Croissant, CDIF, OSI, ODRL, OpenLineage, Grust-ready graph envelope
`lakecat-cli`	`config --catalog URL` and friends
Feature flags	`sail-local`, `typesec-local`, `grust-turso-local`, `turso-local` compose the local integration set

Chapter 6. The Bootstrap Handoff

LakeCat's signature surface for QueryGraph is the **bootstrap bundle** at `/querygraph/v1/bootstrap`: a projection of live catalog tables into Croissant, CDIF, OSI, ODRL, OpenLineage, and a Grust-ready graph envelope.

The wire format and its verification live in a small shared crate, `qglake-bundle`, used by both producer and consumer. qg-rust does not keep a hand-written copy of those types: it deserializes the canonical `QueryGraphBootstrap`, runs LakeCat's own `verify_manifest`, and layers its Cypher import plan on top. The bundle cannot mean two slightly different things on the two sides of the boundary.

Worked example: from catalog to governed import

```
# Terminal 1: the catalog, with Sail planning, TypeSec receipts,
# and Grust projection bound to table transitions.
LAKECAT_BIND_ADDR=127.0.0.1:8181 \
LAKECAT_TURSO_PATH=target/local/catalog.turso \
LAKECAT_GRUST_TURSO_PATH=target/local/catalog-graph.turso \
cargo run -p lakecat-service \
  --features sail-local,typesec-local,grust-turso-local,turso-local

# Terminal 2: standard Iceberg clients see an ordinary REST catalog...
cargo run -p lakecat-cli -- config --catalog http://127.0.0.1:8181

# ...and QueryGraph sees live tables projected into Croissant, CDIF,
# OSI, ODRL, OpenLineage, and a Grust-ready graph envelope.
curl -s http://127.0.0.1:8181/querygraph/v1/bootstrap > bootstrap.json

# qg-rust verifies the bundle with LakeCat's own shared wire crate
# (qglake-bundle) and emits its import plan:
cargo run -- lakecat-import --bundle bootstrap.json --output import-plan.json
```

The import command prints the bundle's verification report; QueryGraph accepts catalog state as *proof*.

API reference

Surface	Essentials
`qglake-bundle` crate	the shared wire format: `QueryGraphBootstrap` and its `verify_manifest`
`LakeCatBootstrapBundle::from_path(path)` (qg-rust)	parse a bundle file
`.import_plan()`	verification report + the Cypher import plan QueryGraph executes

CLI

```
querygraph lakecat-verify
--bundle ...·querygraph
lakecat-import --bundle ...
--output plan.json
```

Chapter 7. Sail and the Cypher Extension

Sail is the Spark-compatible lakehouse engine (from lakehq) that QueryGraph uses as its compute and storage substrate: typed Parquet tables registered in a Spark Connect-compatible server, queryable from PySpark, with the QueryGraph audit trail (`qg_audit`) living alongside the data it audits.

The stack's fork (branch `grust`) adds a **Cypher graph-query extension compiled into Sail itself** — roughly 5,600 lines across twenty files: a Cypher AST in `sail-sql-parser`, graph analysis in `sail-sql-analyzer`, and plan resolution in `sail-plan`, with Spark Connect integration and a design document (`docs/development/graph-extension.md`). Crucially, it reuses Grust's property-graph model and schema validation rather than reimplementing them.

The consequence: the same semantic graph that QueryGraph loads through Grust is `MATCH`-able from any Spark Connect session — no separate graph database in the deployment, no second copy of the graph to drift.

```
# From PySpark, against the Sail server the lakehouse already runs:
spark.sql("MATCH (m:Metric)-[:MEASURED_OVER]->(d:Dataset) "
          "RETURN m.expression, d.source").show()
```

Part II: QueryGraph, the Semantic Layer

The layer that makes the substrate one system. Chapters 8–16 follow the Rust reference implementation; Chapter 17 covers the Python mirror. Everything in this part is exercised by the cross-language contract (Chapter 22).

Chapter 8. Semantic Croissant: What the Data Is

MLCommons Croissant is the ML community's standard for dataset metadata: a JSON-LD document describing a dataset's identity, license, files (`distribution`), record sets, and fields, with `sameAs` links from fields to shared semantic types. It is how Hugging Face, Kaggle, and Google Dataset Search describe ML-ready data — which makes it QueryGraph's front door: a Croissant document is both what QueryGraph *emits* for everything it loads and what it *accepts* from outside (LakeCat bundles, public dataset exports).

QueryGraph generates a Croissant sidecar per dataset, then treats its fields as the atoms of meaning: they become OSI column expressions (Chapter 12), ontology terms, and Grust graph nodes. From a real bundle:

```
{
  "@type": "cr:Dataset",
  "@id": "https://querygraph.ai/datasets/hazards/#dataset",
  "name": "Hazard vocabulary",
  "license": "https://creativecommons.org/licenses/by/4.0/",
  "distribution": [{
    "@type": "cr:FileObject",
    "contentUrl": "https://querygraph.ai/datasets/hazards.csv",
    "encodingFormat": "application/octet-stream"
  }],
  "recordSet": [{
    "@type": "cr:RecordSet",
    "field": [{
      "@type": "cr:Field",
      "name": "subject",
      "dataType": "sc:Text",
      "sameAs": "https://schema.org/about"
    }, "..."]
  }]
}
```

The `sameAs` link is the semantic hook: it says this column *means* something shared, not just something typed.

Chapter 9. CDIF: How the Data Is Found and Accessed

CODATA's Cross-Domain Interoperability Framework covers the FAIR-metadata axes Croissant leaves out: discovery, access, rights, units, vocabularies, and profile declarations, expressed over the DCAT and Dublin Core vocabularies that data catalogs and national research infrastructures speak.

QueryGraph projects every dataset into a CDIF resource that declares which CDIF profiles it satisfies and how to reach the data — and carries the ODRL policy reference, so discovery metadata and rights metadata never separate:

```
{
  "@type": "dcat:Dataset",
  "dct:title": "Hazard vocabulary",
  "cdif:profile": [
    "https://cdif.codata.org/profile/discovery",
    "https://cdif.codata.org/profile/manifest",
    "https://cdif.codata.org/profile/data-description",
    "https://cdif.codata.org/profile/data-access", "..."
  ],
  "dcat:landingPage": "https://querygraph.ai/datasets/hazards",
  "cdif:dataElement": ["...field descriptors..."],
  "dct:accessRights": {"odrl:hasPolicy": ".../policy/default"}
}
```

Croissant answers *what is in the data*; CDIF answers *how a stranger finds, evaluates, and accesses it*. QueryGraph derives the CDIF projection from the Croissant document (`CdifResource::from_croissant`), so the two can never disagree about the dataset they describe.

Chapter 10. DID: Who Is Acting

W3C Decentralized Identifiers give agents, services, and datasets stable identity without a registry. QueryGraph uses two DID methods, each earning its place:

- **`did:oyd`** for deterministic agent and dataset identity: the DID is derived from a seed by hashing, so the same input always yields the same identifier — in Rust and Python, byte for byte (a fixture in the equivalence suite pins `did:oyd:zQmciWcCbpq...` for a known seed). Identity becomes reproducible: re-run the pipeline, get the same actors.
- **`did:key`** for verification: Ed25519 public keys published as self-certifying identifiers (`did:key:z6Mk...`) in envelope `verification_method` fields, resolvable by anyone with no infrastructure.

A generated DID document, from a real bundle:

```
{
  "@context": ["https://www.w3.org/ns/did/v1",
               "https://w3id.org/security/suites/ed25519-2020/v1"],
  "id": "did:oyd:zQmciWcCbpqbsYcNPVzdQ4YqznbAK9kRsnNRDdXg5Z73qCe",
  "controller": "AI Navigator",
  "public_key_multibase": "zFNru6TqKpt4ymk5pbCM5Bq4beqJ9nEDgLucfyv9nr98e",
  "service_endpoint": "https://querygraph.ai/datasets/hazards"
}
```

Textbook rule: DIDs say *who is acting*; ODRL says *what is allowed*; TypeSec turns the decision into a proof the code can carry.

Chapter 11. ODRL: What Is Allowed

The Open Digital Rights Language expresses machine-actionable policy: permissions and prohibitions binding an **action**, an **assignee**, and a **target**, each optionally constrained. QueryGraph's profile uses `odrl:use`, `odrl:read`, `odrl:derive`, and two namespaced actions — `querygraph:translate` and

`querygraph:index`. The evaluation rule is strict: an action is allowed only if a permission matches the assignee and action *and no prohibition does*.

A generated policy — note the shape of the default: the public may read with attribution, the navigator may index, and *nobody* may derive (train on) without a separate agreement:

```
{
  "@type": "odrl:Policy",
  "odrl:target": "https://querygraph.ai/datasets/hazards/#dataset",
  "odrl:assigner": "did:oyd:zQmciWcCbpq...",
  "odrl:permission": [
    {"odrl:action": "odrl:read", "odrl:assignee": "public",
     "odrl:constraint": "attribution required"},
    {"odrl:action": "querygraph:index",
     "odrl:assignee": "did:oyd:zQmciWcCbpq...",
     "odrl:constraint": "local semantic indexing for AI Navigator"}
  ],
  "odrl:prohibition": [
    {"odrl:action": "odrl:derive", "odrl:assignee": "public",
     "odrl:constraint": "no model training without separate agreement"}
  ]
}
```

The four layers ship together as the **Navigator bundle** — one JSON-LD document with `@context` for all four vocabularies — built by the CLI (`navigator`), the `/v1` API, and the MCP tools, identically in both languages (the equivalence suite asserts byte equality modulo timestamps):

```
querygraph navigator \
  --dataset-name "Hazard vocabulary" \
  --description "Controlled vocabulary with multilingual technical terms" \
  --landing-page "https://querygraph.ai/datasets/hazards" \
  --data-url "https://querygraph.ai/datasets/hazards.csv"
# → {"@type": "querygraph:AiNavigatorSemanticBundle",
#    "layers": {"semanticCroissant": ..., "cdif": ..., "did": ..., "odrl": ...}}
```

API reference (the four projections, both languages)

Layer	Rust	Python
Croissant	`CroissantDataset { files, record_sets, ... }`, `Field::new(...).semantic_type(iri)`, `.to_json_ld()`	`croissant.CroissantDataset`, `Field(name, dtype, desc).semantic_type(iri)`, `.to_json_ld()`
CDIF	`CdifResource::from_croissant(dataset, landing, data_url)`, `.with_odrl_policy(id, json)`, `.to_json_ld()`	`cdif.CdifResource` — same constructors

DID	`DidDocument::new_oyd(seed, controller), .with_service_endpoint(url), .to_json()`	`did.DidDocument.new_oyd(seed, controller)` — byte-identical output
ODRL	`Policy { permissions, prohibitions }, Rule { action, assignee, constraint }, Action::{Use, Read, Derive, Translate, Index}, .allows(assignee, action), .to_json_ld()`	`odrl.Policy / Rule / Action` — same evaluation rule
Bundle	`AiNavigator::build(NavigatorInput) -> NavigatorOutput { croissant, cdif, did, odrl, bundle }`	`AiNavigator().build(NavigatorInput(...))`

Chapter 12. OSI: Business Semantics

The Open Semantic Interchange model is where business language meets governed columns. An OSI document carries one semantic model:

- **datasets**, each with a `source` (a governed Sail table like `sail.qg_lakehouse.government_finance__countydata`), primary/unique keys, and **fields**;
- **metrics**, each with an expression per SQL **dialect** — QueryGraph supports `ANSI_SQL`, `SAIL_SQL`, `SNOWFLAKE`, `MDX`, `TABLEAU`, `DATABRICKS`, and `MAQL`, resolving a requested dialect with fallback to `ANSI_SQL` then `SAIL_SQL`;
- **relationships** between datasets (join keys);
- **ontology terms** linking fields to shared vocabularies;
- **`ai_context`** on every level: instructions, synonyms, and examples written *for the LLM* — the model's way of telling an agent how to use it.

Croissant documents project into OSI mechanically: every recordSet field becomes a governed `SAIL_SQL` column expression, `sameAs` types become ontology terms, and a `row_count` metric is attached. The projection is implemented identically in Rust (`OsiDocument::from_croissant_json`, behind `POST /v1/models/import/croissant`) and Python (`OsiDocument.from_croissant`).

Worked example: Croissant becomes OSI, identically in both languages

```
let croissant = serde_json::json!({
    "name": "Energy Burden",
    "description": "Demo energy fields",
    "recordSet": [{"field": [{
        "name": "monthly_cost",
        "description": "Monthly household energy cost",
        "sameAs": "https://querygraph.ai/ontology/monthlyEnergyCost",
    }]}],
});
let osi = OsiDocument::from_croissant_json(&croissant, "qg_lakehouse")?;
assert_eq!(osi.semantic_model.name, "energy_burden_semantic_model");
assert_eq!(osi.semantic_model.datasets[0].source,
```

```
            "sail.qg_lakehouse.energy_burden");
from querygraph.osi import OsiDocument

osi = OsiDocument.from_croissant(dataset)          # same projection rules
osi.semantic_model.resolve_metric("row_count")     # 'COUNT(*)' via SAIL_SQL
osi.semantic_model.find_by_synonym("energy")       # datasets/fields/metrics
```

API reference

Surface	Essentials
`OsiDocument`	`from_mapping` (accepts upstream `semantic_model: []` lists), `from_yaml_file`, `to_json`; Rust adds `from_croissant_json(&value, schema)` and `for_dataverse(&datasets)`; Python adds `from_croissant(dataset)`
`OsiSemanticModel`	`datasets`, `metrics`, `relationships`, `ontology_terms`, `ai_context`; Python: `resolve_metric(name, dialect="SAIL_SQL")` (fallback `ANSI_SQL` → `SAIL_SQL`), `find_by_synonym(term)`
`OsiAiContext`	`instructions`, `synonyms`, `examples`; bare strings coerce to `instructions`
Dialects	`ANSI_SQL`, `SAIL_SQL`, `SNOWFLAKE`, `MDX`, `TABLEAU`, `DATABRICKS`, `MAQL`

Chapter 13. Governance: The Dual Gate

QueryGraph's access decision is deliberately redundant: **RBAC and ODRL must both allow**.

- The RBAC layer is role-based: grants bind principals to roles; permissions bind roles to `(resource, action)` pairs.
- The ODRL layer is policy-based: a permission rule must match the assignee and action, and no prohibition may match.

The two layers fail differently — a fat-fingered role grant does not bypass a prohibition, and a permissive policy does not bypass a missing role — and the combined decision is issued as an **`AccessReceipt`**: principal, resource, action IRI, the boolean, the reason, and the policy id. A denial is a receipt, not an error: it flows through the same channels as an allow, gets signed into the same envelopes, and appears in the same lineage.

```
from querygraph.odrl_rights import OdrlRightsLayer

decision = rights.decide(principal, "sail.qg_lakehouse.finance", Action.READ)
decision.allowed        # rbac_allowed AND odrl_allowed
decision.receipt.reason # "RBAC and ODRL permitted action" / "... denied ..."
```

A real denial receipt, captured from a navigator-loop run (Chapter 21):


```
{
  "principal": "did:example:qg-navigator",
  "resource": "sail.qg_lakehouse.haalsi_baseline__restricted_raw",
  "action": "odrl:read",
  "allowed": false,
  "reason": "RBAC or ODRL denied action",
  "policy_id": "urn:querygraph:policy:navigator-demo"
```

```
}
```

API reference

Surface	Essentials
`RbacPolicy`	`grants: [RoleGrant{principal, role}]`, `permissions: [RolePermission{role, resource, action}]`, `allows`, `roles_for`; Rust adds `decide -> RbacDecision{matched_roles, ...}`
`Policy` (ODRL)	`allows(assignee, action)` — permission must match, no prohibition may
`OdrlRightsLayer`	`decide(principal, resource, action) -> OdrlDecision{rbac_allowed, odrl_allowed, allowed, receipt}`
`AccessReceipt`	`principal`, `resource`, `action` (IRI), `allowed`, `reason`, `policy_id`, `issued_at`
MCP	`check_access(principal, resource, action)` returns the full decision; denials are results

Chapter 14. Lineage and Attestations

Every governed run emits an **OpenLineage** event — the open standard for data lineage that Marquez and the OpenLineage ecosystem consume. QueryGraph's events are COMPLETE run events carrying a custom facet (`queryGraph_typeDid`) that binds the run to the envelope that authorized it: protocol, conversation id, payload hash, signature.

Conformance is proven, not asserted. The official 2-0-2 JSON Schema is vendored in qg-python and validated with format checking on — a discipline that immediately caught a real bug: the spec requires `run.runId` to be a UUID, and both implementations were emitting prefixed hashes. Both now derive run ids as **deterministic UUIDv5** values under a shared namespace (`uuid5(NAMESPACE_URL, "https://querygraph.ai/openlineage")`), from the same seeds as before; a fixture pins the cross-language derivation, and the equivalence suite schema-validates the events both CLIs emit.

Above the events sit **attestations**: the event's canonical hash is signed (Ed25519, `did:key` verification method) into a `LineageAttestation` with a Merkle root, so an auditor can verify that the lineage record is exactly the one the issuer anchored. Events and attestations land in JSONL sinks and in Sail tables (`qg_audit`) next to the data they describe.

Worked example: sign in Python, verify in Rust

```
# Python signs an envelope and writes it out...
uv run python - <<'PY'
import json
from querygraph.typedid import TypeDidAgent
env = TypeDidAgent.new("SupervisorAgent").request(
    TypeDidAgent.new("FinanceAgent"),
    action="summarize", resource="compartment:finance",
    payload={"question": "Where is fiscal stress highest?"})
open("envelope.json", "w").write(env.model_dump_json())
PY

# ...and the Rust CLI verifies it with no shared state:
cargo run -- verify-envelope --file envelope.json
```

```
{
  "payload_hash_valid": true,
  "signed": true,
  "signature_valid": true,
  "verification_method": "did:key:z6MkrdhpoFnCtEGK3RhXqryfjxVpy...",
  "scheme": "ed25519"
}
```

Rust resolves the `did:key`, reconstructs the documented signing payload (`querygraph-typedid-signing-v1`), and recomputes Python's canonical JSON byte-for-byte — `sort_keys`, compact separators, `ensure_ascii` escapes and all. Change one byte anywhere and `signature_valid` flips to `false` with a non-zero exit. The same check is served at `POST /v1/audit/verify-envelope` and as the `verify_envelope` MCP tool in both languages.

API reference

Surface	Essentials
`OpenLineageRunEvent`	`for_agent_run(request=envelope, job_name, inputs, outputs)` (Python) / `for_dataverse_agent_run(...)` (Rust); `event_hash()` — canonical sha256
`run_id_for(seed)`	deterministic UUIDv5 under `uuid5(NAMESPACE_URL, "https://querygraph.ai/openlineage")` — identical in both languages
`LineageAttestation`	`from_event(issuer, subject, event_hash, signer=...)`, `verify()`, `signing_payload()` (`querygraph-lineage-attestation-v1`)
Validation (Python)	`validation.validate_openlineage_schema(event)` — vendored official 2-0-2 schema, format-checked; `validate_croissant`, `validate_cdif`, `validate_openlineage` shape checks
Sinks	`append_jsonl(path, event)`; Sail `qg_audit` tables; Rust `--openlineage-{file,url,sail}`

Chapter 15. The Lakehouse Path

The data side of the reference implementation:

- **Loaders** stream CSV/TSV/XLSX into typed Parquet tables in Sail, registering views and writing a load manifest. Types are inferred, columns normalized to safe SQL names.
- **The Dataverse end-to-end path** (`dataverse-e2e`) takes live Dataverse datasets through the whole chain — fetch, stage into Sail, project Croissant/CDIF/OSI, gate with RBAC+ODRL, wrap the question in a TypeDID envelope, optionally **call Ollama through a DID-encrypted prompt-bound gateway** (`--call-ollama`), and emit lineage. The `--live-sail` flag runs it against a real Sail server.
- **The demonstration corpus** both books use: county government finance, household energy access, dockless-transportation trips, climate-health pathways, CODATA physical constants, and a *restricted* health baseline that exists to be denied.

- From Python, `querygraph lakehouse-register` and `audit-register` attach the loaded tables and the audit trail to a PySpark session over Spark Connect.

```
spark.sql("SELECT COUNT(*) FROM global_temp.government_finance__countydata")
spark.sql("SELECT event_hash, job_name FROM global_temp.openlineage_events")
```

Chapter 16. The QGLake Story: The Resilience Desk

The stack's canonical governed multi-agent narrative, implemented deterministically in both languages — the golden baseline the live loop is measured against:

- A **supervisor** takes the question ("Where do fiscal capacity, energy burden, mobility disruption, and climate-health risk overlap?") and delegates to six **compartmentalized specialists** — finance, energy, mobility, climate-health, reference data, and a restricted-data broker — each holding capabilities scoped to its own compartment (`CanReadDatasetCompartment`, `CanDeriveRedactedSummary`, ...).
- Specialists never share raw rows. Each returns a **signed summary** in a TypeDID envelope.
- The **restricted-data broker is denied**: it returns a signed metadata-only denial receipt instead of restricted health rows — the pattern that makes denial a first-class result.
- A **synthesis agent** sees only the signed summaries (capability: `CanAggregateSummaries`), never the compartments.
- The run emits an OpenLineage event over all consulted (and denied) sources, anchored by an Ed25519 attestation.

Run it: `cargo run -- qglake-story --json` or `python -m querygraph qglake-story --pretty`. The equivalence suite asserts the two implementations agree on the governance semantics: same roster, the broker (and only it) denied, attestation schemas field-for-field identical.

API reference

Surface	Essentials
Rust	`run_qglake_story() -> QgLakeStoryReport` (title, question, supervisor, specialists, synthesis, rbac, policies, semantic_catalog, typesec, open_lineage, did_attestation); `render_qglake_story(&report)` for the readable briefing
Python	`qglake.build_python_qglake_story() -> dict` (prompt, agents, responses, synthesis, openlineage, attestation)
CLI	`qglake-story --json` (Rust) · `qglake-story --pretty` (Python); also `GET /v1/qglake/story` and the `run_qglake_story` MCP tool

Chapter 17. qg-python: The Pythonic Mirror

`qg-python` mirrors the same layers Pydantic-v2-first — every model above is a `BaseModel` with the same field names — and adds the ecosystem surfaces Python is for:

- **Real crypto** (`crypto` extra): Ed25519 signing with seed-derived keys, `did:key` verification methods, envelope and attestation verification. Without the extra, digests are labeled `unsigned:sha256:` — never mistakable for signatures.
- **The governed navigator loop** (Chapter 21) and **`api_auth`** (Chapter 18's client side).
- **MCP server** (`mcp` extra) and **A2A card** (Chapters 19–20).
- **Framework adapters**: LangChain `StructuredTool` (sync and async), vendor-neutral `to_tool_schema()`.

- **Lakehouse helpers**: PySpark against Sail's Spark Connect endpoint.

Install: `pip install querygraph` (core is pure Pydantic), with extras `crypto`, `mcp`, `agents`, `validation`, `lakehouse`, or `all`. The package ships `py.typed`; the CLI mirrors the Rust commands.

The division of labor: Rust loads and verifies the warehouse and serves the platform; Python gives notebooks, PySpark users, and agent frameworks a typed interop layer — and the two are held identical where they overlap (Chapter 22).

API reference (package map)

Module	Provides
`querygraph.crypto`	`Ed25519Signer.from_seed/.generate/.sign/.did_key/.verification_method`, `verify(key, msg, sig)`, `public_key_from_did_key`, `unsigned_digest`, `CRYPTO_AVAILABLE`
`querygraph.typedid`	`TypeDidAgent.new(name, seed=...)/.request/.answer/.signer/.did_key/.to_tool_schema`, `TypeDidEnvelope.create/.verify_payload/.verify_signature/.is_signed`, `AccessReceipt`, `GovernedPrompt`, `AgentResponse`, `signing_payload_v1`
`querygraph.navigator_loop`	`GovernedNavigatorLoop(document, rights, llm=..., principal=...)`, `.answer(question) -> NavigatorAnswer`, `.demo()`, `openai_compatible_llm(base_url, model, api_key=...)`
`querygraph.api_auth`	`mint_envelope_header(agent, path=..., body=...)`, `governed_post(base, path, payload, agent)`
`querygraph.mcp_server`	`create_server(osi_path=..., rights_path=...)`, `serve(transport=...)`, `demo_rights_layer`, `load_rights_layer`, `parse_action`
`querygraph.a2a`	`build_agent_card(base_url)`, `SKILLS`, `A2A_PROTOCOL_VERSION`
`querygraph.agents`	`TypeDidLangChainToolAdapter(.invoke/.ainvoke/.as_tool/.as_async_tool)`, `to_tool_schema(agent, flavor=...)`, `deterministic_specialist`, `TypeDidAgentRun.aggregate`
`querygraph.lakehouse` / `lineage` / `osi` / `croissant` / `cdif` / `did` / `odrl` / `odrl_rights` / `rbac` / `validation`	mirrors of the Rust layers (Chapters 8–15)

Extras: `crypto`, `mcp`, `agents`, `validation`, `lakehouse`, `all`; the package ships `py.typed`.

Part III: The Interoperability Surfaces

QueryGraph does not compete with agent frameworks — it is the governed data and semantics plane they plug into. Goshawk built these doors; Sentinel guards them.

Chapter 18. The /v1 API and Envelope Auth

`querygraph serve --port 8080` exposes the platform over HTTP:

Route	What
`GET /v1/health`	service, API version, release
`POST /v1/navigator/bundle`	the four-layer semantic bundle
`GET /v1/qglake/story`	the Resilience Desk with full evidence chain
`POST /v1/audit/verify-envelope`	envelope verification (a bad signature is a 200 + receipt, not a 5xx)
`POST /v1/models/import/{osi,croissant}`	semantic-model registry imports
`GET /v1/models`, `GET /v1/models/{name}`	registry listing and fetch
`GET /v1/search?q=`	search names, descriptions, ai_context, semantic types, ontology terms
`POST /v1/answer`	search → plan → synthesize → sign (Chapter 21)
`GET /.well-known/agent-card.json`	the A2A card (Chapter 20)

With `--require-auth`, the governed routes (`models/import/*`, `answer`) demand a signed TypeDID envelope in the **`x-qg-envelope`** header. The contract binds three things: the action must be `invoke`; the resource must equal the request path (no cross-route replay); and the payload must carry `bodySha256`, the hash of the exact request body (no body swapping). The signature verifies against the envelope's own `did:key` verification method.

Worked example: the guarded /v1, from both sides

An unauthenticated request gets a 401 whose receipt *teaches the contract* (captured from a live server):

```
$ curl -s -X POST http://127.0.0.1:8080/v1/answer \
    -H 'content-type: application/json' -d '{"question":"?"}'
{
  "error": "missing x-qg-envelope header",
  "receipt": {
    "allowed": false,
    "path": "/v1/answer",
    "contract": {
      "header": "x-qg-envelope",
      "action": "invoke",
      "resource": "<request path>",
      "payload": {"bodySha256": "<sha256 hex of request body>"},
      "signature": "ed25519 over querygraph-typedid-signing-v1"
    }
  }
}
```

The Python client satisfies it in two lines:

```
from querygraph.api_auth import governed_post
from querygraph.typedid import TypeDidAgent

result = governed_post(
    "http://127.0.0.1:8080", "/v1/answer",
    {"question": "what is fiscal capacity?"},
    TypeDidAgent.new("ApiClient"),
)
```

The equivalence suite proves this live: it boots `qg-rust serve --require-auth`, posts without the header (asserting the 401 receipt), then with a Python-minted header (asserting the governed answer).

API reference

`querygraph serve --port 8080 [--require-auth]`. The `x-qg-envelope` header is a compact-JSON TypeDID envelope; the middleware checks, in order: signature validity (against the envelope's `did:key verification_method`), `action == "invoke"`, `resource == <request path>`, and `payload.bodySha256 == sha256(body)`. Failure returns `401 {error, receipt: {allowed: false, path, checks, contract}}`. Clients: `querygraph.api_auth.mint_envelope_header` / `governed_post`.

Chapter 19. MCP: One Server, Every Framework

The Model Context Protocol is how agent frameworks discover tools in 2026, and the stack speaks it from both languages:

- **Python** (`querygraph mcp-serve`, official SDK): tools `search_semantic_model`, `resolve_metric` (dialect fallback), `check_access` (the dual gate — denials are receipts), `build_navigator_bundle`, `run_qglake_story`, `verify_envelope`, and `answer_question`; resources `qg://story/resilience-desk` and `qg://models/current`; stdio, SSE, and streamable-HTTP transports; `--osi` loads your model, `--rights` your governance JSON.
- **Rust** (`querygraph mcp-serve`): a dependency-free JSON-RPC 2.0 implementation of the MCP handshake (protocol 2024-11-05) over stdio, with the same governed surface, sharing the `/v1` internals (the answer core is literally the same function).

Any MCP client — Claude Code/Desktop, LangChain via `langchain-mcp-adapters`, PydanticAI, LlamaIndex, CrewAI, the OpenAI Agents SDK — reaches everything with zero adapter code.

Worked example: an MCP session, by hand

```
$ printf '%s\n%s\n%s\n' \
  '{"jsonrpc":"2.0","id":1,"method":"initialize","params":{}}' \
  '{"jsonrpc":"2.0","method":"notifications/initialized"}' \
  '{"jsonrpc":"2.0","id":2,"method":"tools/list"}' \
  | querygraph mcp-serve
{"id": 1, "result": {"protocolVersion": "2024-11-05",
  "serverInfo": {"name": "querygraph", "version": "0.4.0"}, ...}}
{"id": 2, "result": {"tools": ["build_navigator_bundle",
  "run_qglake_story", "verify_envelope", "import_semantic_model",
  "search_semantic_models", "answer_question"]}}
```

API reference (tools)

Tool	Arguments	Returns
`search_semantic_model` (py) / `search_semantic_models` (rs)	`term`	matches: kind/name/dataset
`resolve_metric` (py)	`name`, `dialect="SAIL_SQL"`	expression, with dialect fallback
`check_access` (py)	`principal`, `resource`, `action`	the full dual-gate decision + receipt
`build_navigator_bundle`	dataset name/description/landing/data URL/creator/agent	the four-layer bundle

`run_qglake_story`	—	the Resilience Desk report
`verify_envelope`	`envelope`	verification report
`import_semantic_model` (rs)	`osi` or `croissant` document	import summary
`answer_question`	`question`	answer + plans + envelope (+ receipts in py)

Python resources: `qg://story/resilience-desk`, `qg://models/current`. Transports: stdio (both), SSE and streamable-HTTP (Python).

Chapter 20. A2A, Tool Schemas, and Adapters

The Agent Card. Both implementations publish the same Linux Foundation Agent2Agent v0.3.0 card — served at `/.well-known/agent-card.json`, printed by `agent-card` — declaring five skills (`navigator-bundle`, `qglake-story`, `verify-envelope`, `import-semantic-model`, `semantic-search`) and a security scheme that documents the TypeDID envelope contract. The card is a cross-language contract: the equivalence suite asserts the two implementations publish identical skills, capabilities, and security schemes.

Tool schemas. For function-calling runtimes rather than MCP, one agent exports both flavors — accepted by OpenAI, Anthropic, Mistral, vLLM, Ollama, and most local servers:

```
agent = TypeDidAgent.new("FinanceAgent")
agent.to_tool_schema()                      # {"type": "function", "function":
                                            #  {"name": "FinanceAgent",
                                            #   "parameters": {...}}}   ← OpenAI
agent.to_tool_schema(flavor="anthropic")    # {"name": "FinanceAgent",
                                            #  "input_schema": {...}}   ← Anthropic
```

LangChain. `TypeDidLangChainToolAdapter` wraps a governed agent as a `StructuredTool` — `as_tool()` for sync runtimes, `as_async_tool()` for async ones. Every adapter result carries the signed envelope with the answer; nothing hands back a bare string.

Chapter 21. The Governed Navigator Loop

Sentinel's centerpiece: the loop that turns a question into a governed, signed, lineage-anchored answer.

1. **Search** the semantic model — names, synonyms (including multi-word synonyms via bigrams), descriptions, `ai_context`.
2. **Gate** every matched dataset through RBAC+ODRL; collect receipts; denied sources go into `denied_sources` and are *named in the prompt as off-limits* — never planned, never touched.
3. **Plan** SQL over allowed Sail sources only, resolving matched metrics with dialect fallback.
4. **Synthesize** with any `Callable[[str], str]` — the `openai_compatible_llm` helper binds Ollama, vLLM, llama.cpp, LM Studio, or OpenRouter — or deterministically when no model is given. Same governance either way; the deterministic path is the golden baseline.
5. **Sign and record**: the answer, plans, and receipts travel in a signed TypeDID envelope; a schema-valid OpenLineage event and an Ed25519 attestation complete the chain.

The Rust side serves the deterministic core as `POST /v1/answer` and the `answer_question` MCP tool (one shared function).

Worked example: the loop, receipts and all

```
from querygraph.navigator_loop import GovernedNavigatorLoop

loop = GovernedNavigatorLoop.demo()   # or (your_osi_doc, your_rights, llm=...)
result = loop.answer(
    "Where do fiscal capacity and energy burden overlap with health risk?"
)
```

Captured from a real run:

```
{
  "answer": "Answerable from governed sources
     sail.qg_lakehouse.access_2018__access_data,
     sail.qg_lakehouse.government_finance__countydata via 3 planned
     queries. Restricted sources were denied with receipts:
     sail.qg_lakehouse.haalsi_baseline__restricted_raw.",
  "denied_sources": ["sail.qg_lakehouse.haalsi_baseline__restricted_raw"],
  "plans": [
    "SELECT * FROM sail.qg_lakehouse.government_finance__countydata",
    "SELECT `monthly_cost` FROM sail.qg_lakehouse.access_2018__access_data",
    "SELECT SUM(total_revenue - mandated_spend)
       FROM sail.qg_lakehouse.government_finance__countydata"
  ]
}
```

Swap in a live model with one callable — the governance is unchanged:

```
from querygraph.navigator_loop import openai_compatible_llm

loop = GovernedNavigatorLoop.demo(
    llm=openai_compatible_llm("http://localhost:11434", "llama3.2"),
    llm_name="ollama:llama3.2",
)
```

API reference

Surface	Essentials
`GovernedNavigatorLoop(document, rights, llm=None, llm_name=..., agent_name=..., principal=...)`	the loop; `llm: Callable[[str], str]`
`.answer(question) -> NavigatorAnswer`	`answer`, `synthesized_by`, `matches`, `plans: [PlannedQuery{dataset, source, sql, metric}]`, `receipts`, `denied_sources`, `envelope`, `openlineage`, `attestation`
`GovernedNavigatorLoop.demo(llm=..., llm_name=...)`	the Resilience Desk demo configuration
`openai_compatible_llm(base_url, model, api_key=None)`	binds `/v1/chat/completions` — Ollama, vLLM, llama.cpp, LM Studio, OpenRouter
Rust	`POST /v1/answer {question}`; `answer_question` MCP tool; shared `answer_over_models` core

Chapter 22. The Cross-Language Contract

Rust and Python are held equivalent by an executable contract (`qg-python/tests/test_rust_equivalence.py`), not by discipline:

- **Bundles**: `navigator` output is byte-identical modulo timestamps.
- **The story**: same specialist roster, the restricted broker (and only it) denied, COMPLETE OpenLineage events, attestation schemas field-for-field identical.
- **Crypto, both directions**: Python signs → the Rust CLI verifies and rejects a tampered copy with exit 1; Rust mints envelopes from the same seeds and a fixture pins the shared `did:key`.

- **Lineage**: both CLIs' events validate against the official OpenLineage schema; a fixture pins the shared UUIDv5 run-id derivation (`run_id_for("test")` is the same UUID in both languages).
- **A2A**: both agent cards publish identical skills, capabilities, and security schemes.
- **Auth, live**: a running `qg-rust serve --require-auth` rejects a bare POST (401 + receipt) and accepts a Python-minted header (200 + answer).

The suite runs 49 Python tests (12 at the start of the interoperability work) alongside 40 Rust tests, in CI on every push.

Part IV: Integration in Practice

Chapter 23. Assembling the Lakehouse in Ten Steps

The dedicated book walks this assembly in depth; here is the whole system in one sitting — the sequence the `dataverse-e2e` and lakehouse commands automate, each step naming the component that owns it:

1. **Load the lakehouse** (Sail). Stream CSV/TSV/XLSX — or live Dataverse datasets — into typed Parquet tables; register views; write the load manifest.
2. **Materialize Semantic Croissant** (Chapter 8). A JSON-LD sidecar per dataset: files, record sets, fields, semantic types.
3. **Project CDIF** (Chapter 9). Discovery, access, and profile metadata derived from the Croissant document — never hand-maintained.
4. **Build the OSI semantic model** (Chapter 12). Fields become governed `SAIL_SQL` expressions; metrics get per-dialect SQL; ontology terms link shared meaning.
5. **Load the Grust graph** (Chapters 1–2). Datasets, fields, terms, agents, and policies become nodes and edges — queryable by Cypher, including from Spark via the Sail extension.
6. **Identify agents with DIDs** (Chapter 10). Deterministic `did:oyd` documents from seeds; the same seed yields the same identity in both languages.
7. **Apply ODRL rights** (Chapter 11). Permissions and prohibitions per dataset; the read/index/derive defaults that make training-by-default impossible.
8. **Mint TypeSec capabilities** (Chapter 3). `CanReadDatasetCompartment`, `CanDeriveRedactedSummary`, `CanAggregateSummaries`, ... — proofs, not flags.
9. **Route to compartmentalized agents** (Chapter 16). Supervisor → specialists → restricted broker → synthesis; raw rows never cross compartments; denials are signed receipts.
10. **Call the model through TypeDID** (Chapters 4, 21). The prompt travels DID-encrypted and prompt-bound to Ollama (or any OpenAI-compatible endpoint via the loop); the reply comes back in a signed envelope; the run lands in OpenLineage with an Ed25519 attestation.

```
# The compressed version of all ten:
cargo run -- dataverse-e2e --live-sail --call-ollama \
  --question "Which governed datasets are relevant?"
```

Chapter 24. End to End: Catalog to Governed Answer

The full path, using only released commands:

```
# 1. Start the platform.
cargo run -- serve --port 8080 --require-auth

# 2. Import semantics — from a Croissant document (or LakeCat's
#    bootstrap projection, Chapter 6, or an OSI YAML).
python - <<'PY'
from querygraph.api_auth import governed_post
from querygraph.typedid import TypeDidAgent
agent = TypeDidAgent.new("ApiClient")
```

```
croissant = {
    "name": "Energy Burden",
    "recordSet": [{"field": [
        {"name": "monthly_cost", "description": "Monthly energy cost"}]}],
}
print(governed_post("http://127.0.0.1:8080",
                    "/v1/models/import/croissant", croissant, agent))
# {'imported': 'energy_burden_semantic_model', 'datasets': 1, ...}
PY

# 3. Search it — open route, no auth needed for reads.
curl -s 'http://127.0.0.1:8080/v1/search?q=monthly'
# {"matches": [{"kind": "field", "name": "monthly_cost", ...}]}

# 4. Ask — governed route, signed envelope required.
python - <<'PY'
from querygraph.api_auth import governed_post
from querygraph.typedid import TypeDidAgent
result = governed_post("http://127.0.0.1:8080", "/v1/answer",
    {"question": "What drives monthly energy burden?"},
    TypeDidAgent.new("ApiClient"))
print(result["answer"])          # planned over sail.qg_lakehouse.energy_burden
print(result["envelope"]["payload_sha256"][:16])  # ...and it's signed
PY

# 5. Verify the answer's envelope — anyone can, with no shared state.
curl -s -X POST http://127.0.0.1:8080/v1/audit/verify-envelope \
  -H 'content-type: application/json' -d @answer-envelope.json
```

Every hop leaves evidence: the import is authorized by a path-bound envelope, the answer carries its plans and receipts, and the verification endpoint lets a third party check the signature.

Chapter 25. Live Dataverse to Governed Answer

The second integration walkthrough runs the whole chain over *live* research data: Harvard Dataverse, the largest public Dataverse instance. One command — every step of Chapter 23, against data QueryGraph has never seen:

```
cargo run -- dataverse-e2e \
  --dataverse-url https://dataverse.harvard.edu \
  --query "municipal finance" --limit 2 \
  --question "Which governed datasets describe municipal fiscal capacity?" \
  --openlineage-file lineage.jsonl --did-ledger-file did-ledger.jsonl
```

What happened, from a real run:

1. Live search and staging. The Dataverse Search API returned two real datasets — *Graduate School Rankings for Public Administration Programs* (`doi:10.7910/DVN/YTAB7V`) and *Data Files for Criminal Municipal Courts* (`doi:10.7910/DVN/AXNUVP`) — whose metadata and file listings were staged into Sail as typed views (`dataverse_7424459_metadata`, `dataverse_7424459_files`, ...).

2. Semantics, derived not hand-written. Each dataset projected to Croissant; an OSI model (`querygraph_dataverse_navigator`) was built over both, with ontology terms from their subjects and keywords; the four-layer bundle was generated for the first dataset.

3. The dual gate, with a receipt. The agent's `answer` action passed RBAC (role `navigator` matched) *and* ODRL, and the receipt says so:

```
{
  "action": "answer",
  "allowed": true,
```

```
  "principal": "did:oyd:zQmX7Cm5eikMzwMavBT4zoTUAsWxK3zGwqSbFHu729gb41r",
  "rbac": {"allowed": true, "matched_roles": ["navigator"]},
  "odrl_allowed": true,
  "reason": "TypeSec capabilities, RBAC role assignment, and ODRL
            policy allow the answer path"
}
```

4. The signed request. The governed prompt — question plus the staged metadata, nothing else — traveled in a `typedid/a2a` envelope (`signature: a16f2cba...`), ready for `--call-ollama` to carry it, DID-encrypted and prompt-bound, to a local model.

5. Lineage over DOIs. The OpenLineage event lists the *DOIs as inputs* (`doi:10.7910/DVN/YTAB7V`, `doi:10.7910/DVN/AXNUVP`), carries the semantic bundle hash as a job facet, gets a spec-conformant run id (`f0539cd9-853f-589b-a70f-a9371f532c20` — UUIDv5, derived from the envelope signature), lands in `lineage.jsonl`, and is anchored by an Ed25519 attestation whose issuer is a resolvable `did:key`.

Add `--live-sail --sail-endpoint http://127.0.0.1:50051` to execute against a running Sail server (views become queryable from PySpark), `--call-ollama --ollama-model llama3.2` for live synthesis through the TypeDID gateway, `--anchor-codata` to anchor the bundle DID with the CODATA ODRL service, and `--openlineage-url http://localhost:5000` to emit straight into Marquez.

The point of this walkthrough: nothing above was fixture data. The evidence chain — receipts, envelopes, DOI-level lineage, attestation — assembled itself around unfamiliar, live research data with one command.

Chapter 26. Plugging In Agent Frameworks

Claude Code / Desktop (MCP, stdio). Point the client at either server:

```
{"mcpServers": {"querygraph": {
    "command": "querygraph", "args": ["mcp-serve", "--osi", "model.yaml"]}}}
```

LangChain. Either consume the MCP server via `langchain-mcp-adapters`, or wrap governed agents natively:

```
from querygraph.agents import TypeDidLangChainToolAdapter, deterministic_specialist
from querygraph.typedid import TypeDidAgent

finance = TypeDidAgent.new("FinanceAgent")
adapter = TypeDidLangChainToolAdapter(
    finance, deterministic_specialist(finance, summary="Fiscal summary..."))
tool = adapter.as_async_tool()          # a StructuredTool; results carry envelopes
```

PydanticAI / OpenAI Agents SDK / LlamaIndex / CrewAI. All speak MCP — `querygraph mcp-serve` is the one integration. For direct function-calling, export schemas with `to_tool_schema()` (Chapter 20).

Local and hosted LLMs. The navigator loop binds any OpenAI-compatible endpoint (`openai_compatible_llm(base_url, model)`); the Rust Ollama path wraps calls in DID-encrypted, prompt-bound envelopes so even the inference call is attributable.

Your own governance. Both the MCP server and the loop accept your OSI model (`--osi`) and your RBAC+ODRL policy (`--rights governance.json` — `{"rbac": {...}, "odrl": {...}}`); the demo policies are defaults, not assumptions.

Chapter 27. Operating and Releasing

Build and test.

```
cd qg-rust && cargo test          # 40 tests; clippy -D warnings in CI
cd qg-python && uv sync --extra all && uv run pytest   # 49 tests,
                                  # including the live cross-language contract
```

Run. `serve` (with `--require-auth` in anything shared), `mcp-serve`, `qglake-story`, `answer`, `navigator`, `verify-envelope`, `lakecat-import`, `dataverse-e2e --live-sail --call-ollama` for the full kitchen sink.

CI. GitHub Actions in every repo; qg-rust's workflow assembles the sibling `grust/lakecat` layout to satisfy the path dependencies; qg-python builds the wheel and `twine checks` it on a 3.11/3.13 matrix.

Release discipline. SemVer with codename pools per repo (birds of prey, Venetian landmarks, wild cats); coordinated stack waves; CHANGELOGs and RELEASES logs; versioned book artifacts (`stem (version-hash).{epub,pdf}`) published per release; GitHub releases with attached wheels.

Documentation. Two books in `qg-rust/docs` — the dedicated QueryGraph book (`book/`) and this guide (`guide/`) — plus the review deck (`slides/`) and the one-pager in HTML, typst, and troff (`onepager/`), all rebuilt at each release hash.

API reference (the CLIs)

`qg-rust` (`cargo run -- <command>` or the `querygraph` binary):

Command	What
`navigator`	build the four-layer bundle
`anchor-url`	CODATA ODRL URL→DID anchoring
`dataverse-e2e`	the live chain (Chapter 25); `--live-sail`, `--call-ollama`, `--anchor-codata`, `--openlineage-{file,url,sail}`
`lakehouse-load` / `lakehouse-verify` / `lakehouse-validate`	stream files into typed Sail Parquet; verify and validate
`lakecat-verify` / `lakecat-import`	bootstrap-bundle verification and import planning
`qglake-story [--json]`	the Resilience Desk
`verify-envelope --file`	envelope verification (exit 1 on failure)
`serve [--port] [--require-auth]`	the `/v1` API + agent card
`agent-card [--base-url]`	print the A2A card
`mcp-serve`	MCP over stdio

`qg-python` (`querygraph <command>`): `navigator`, `anchor-url`, `qglake-story`, `lakehouse-register`, `audit-register`, `pyspark-examples`, `answer` (`--osi`, `--rights`, `--llm-base-url`, `--llm-model`), `agent-card`, `mcp-serve` (`--osi`, `--rights`, `--transport`).

Future Work

The near line (post-Sentinel):

- **The live navigator loop as the default path**: LLM runs under identical receipts, benchmarked against the deterministic baseline; Rust parity for the full loop (search and plan are shared today; the LLM binding is Python-first).
- **The remaining `/v1` surface**: lineage event queries, audit verification listings, access *explanation* (why was this denied?) — all behind envelope auth.
- **Adopting Torcello**: TypeSec's interop plane, `mcp-gate` in front of QueryGraph's own MCP servers, signed decision receipts unified with QueryGraph's access receipts, and the enforcement proxy as the governed inference layer. The dependency bump is done; the integration is the work.

The wider arc (from the workspace review, FABLE-REVIEW-1 in the meta-repo):

- **Catalog ecosystem**: Apache Polaris `SemanticModel` entities and a `/navigator-bundle` projection endpoint — LakeCat-first, then the upstream conversation; ODS packaging under `/.well-known/`.
- **Standards round-trips**: `OSIMetricFacet` upstreaming to OpenLineage; Marquez in CI; `mlcroissant` validation; importers from dbt MetricFlow and Cube into OSI; a Hugging Face Croissant importer.
- **Distribution**: crates.io publication once the path-dependency knot is cut; an ADBC/Flight SQL path so notebooks can query Sail without the PySpark stack; a `docker compose up` demo of the whole evidence chain.

- **Scale and research**: Merkle-batched attestations per tenant and time window; OWL/SKOS ontology import into the Grust graph; cross-node federation with inbound signed bundles; and the benchmark that motivates the whole stack — *how much does a governed semantic layer improve agent accuracy over the same lakehouse?* — measured on text-to-SQL tasks with and without OSI/Croissant context.

Appendix: Glossary and Link Index

Glossary

- **Agent Card (A2A)** — a discoverable JSON document describing an agent's skills and security requirements, served at `/.well-known/agent-card.json`.
- **Attestation** — an Ed25519 signature over a lineage event's canonical hash, binding the audit record to its issuer.
- **Bootstrap bundle** — LakeCat's projection of live catalog tables into the QueryGraph semantic formats, verified by the shared `qglake-bundle` crate.
- **Capability** — TypeSec's unforgeable, typed proof that a policy check passed: `Capability<Permission, Resource>`.
- **Dual gate** — QueryGraph's access rule: RBAC *and* ODRL must both allow.
- **Envelope (TypeDID)** — a signed agent message: sender, recipient, action, resource, canonically-hashed payload, Ed25519 signature, `did:key` verification method.
- **Navigator bundle** — the four-layer JSON-LD projection of one dataset: Semantic Croissant + CDIF + DID + ODRL.
- **OSI semantic model** — business terms (datasets, fields, metrics, relationships, ontology terms) with per-dialect SQL and LLM-facing `ai_context`.
- **Receipt** — the recorded outcome of a policy decision, allow or deny, with principal, resource, action, reason, and policy id.
- **Run id** — a deterministic UUIDv5 under the QueryGraph namespace, identical across languages for the same seed.
- **Signing payload** — the documented byte string a signature covers (`querygraph-typedid-signing-v1` for envelopes, `querygraph-lineage-attestation-v1` for attestations).

Link Index

- Meta-repo: https://github.com/querygraph/querygraph
- qg-rust: https://github.com/querygraph/qg-rust · release https://github.com/querygraph/qg-rust/releases/tag/v0.4.0
- qg-python: https://github.com/querygraph/qg-python · release https://github.com/querygraph/qg-python/releases/tag/v0.4.0
- Grust: https://github.com/querygraph/grust
- TypeSec: https://github.com/querygraph/typesec
- LakeCat: https://github.com/querygraph/lakecat
- Sail upstream: https://github.com/lakehq/sail
- Standards: Croissant https://mlcommons.org/croissant/ · CDIF https://cdif.codata.org/ · DID https://www.w3.org/TR/did-core/ · ODRL https://www.w3.org/TR/odrl-model/ · OSI https://github.com/open-semantic-interchange · OpenLineage https://openlineage.io/ · MCP https://modelcontextprotocol.io/ · A2A https://github.com/a2aproject/A2A